Regular Expressions

CSCI 1101

2024-04-26

Embedded Domain-Specific Languages

An **embedded domain-specific language** (EDSL) is a programming language *within* another programming language, used for performing a specialized task.

We have already seen some examples:

string interpolation is an EDSL for concatenating string literals with formatted string representations of other data,

arithmetic operators are an EDSL for forming compound arithmetic expressions.

Regular Expressions

Regular expressions (REs) are an EDSL for specifying strings that conform to certain simple patterns.

The main ways of building regular expressions are:

alternation: choosing one of several alternatives,

iteration: repeating something some number of times.

Each regular expression is a *pattern* representing a *set* of strings.

Specifying Regular Expressions

In order to use regular expressions in python we need to *import* a module:

```
import re
```

In python regular expressions are encoded as strings.

Singleton Regular Expressions

A **singleton regular expression** is a regular expression that specifies exactly one string.

Except when we need to use some escape encoding (discussed shortly), we write a singleton regular expression as the string that it specifies:

```
just_cat = 'cat'  # represents the set {'cat'}
just_hat = 'hat'  # represents the set {'hat'}
```

Matching Regular Expressions

Often we want to know whether and where a string described by a regular expression occurs in another string.

The python `re` module includes several tools for doing this, including:

- `re.search` takes a regular expression and a string and returns an `re.Match` object representing the first match for the regular expression found in the string. It returns `None` if no match was found.
- `re.findall` takes a regular expression and a string and returns a `list` of substrings matching the regular expression.
- `re.finditer` takes a regular expression and a string and returns an *iterator* of matches of the regular expression on the string.

```
> re.search(just_cat , "the cat in the hat")
<re.Match object; span=(4, 7), match='cat'>
```

Alternation

The first way to build compound regular expressions out of simpler ones is by **alternation**.

To specify a regular expression that will match one of several *alternatives*, separate them with a vertical bar `|` without any intervening whitespace.

```
cat_or_hat = 'cat|hat' # represents the set {'cat', 'hat'}


> re.findall(cat_or_hat , "the cat in the hat")
['cat', 'hat']
```

Activity: Digits

Use *alternation* to define a regular expression `digit` that will match any Arabic numeral.

For example:

```
> re.findall(digit , "R2D2")
['2', '2']
> re.findall(digit , "C3P0")
['3', '0']
```

Character Classes

A **character class** is a set of characters that are logically related somehow, for example, digits.

Character classes are so common that they have a special syntax.

We concatenate all of the characters in the class together between braces `[` and `]`, with no separating spaces or commas between them:

```
digit = '[0123456789]'  # a character class for digits
```

Complement Character Classes

The **complement** of a set of character contains those characters *not* in the specified set.

To specify the *complement* of a set of characters, put the special symbol `^` immediately after the opening `[` of a character class:

```
not_digit = '[^0123456789]'   # a class for non-digits


> re.findall(not_digit , "R2D2")
['R', 'D']
> re.findall(not_digit , "C3P0")
['C', 'P']
```

Character Ranges

Characters form a **range** if their encodings are given by consecutive numbers.

This is really only useful for numerals and alphabetic letters, where (for English, anyway) the encodings follow the conventional alphabetic order.

To specify a *range* of characters, put the special symbol `-` between the first and last characters in the range:

```
letter = '[a-zA-Z]'
```

```
> re.findall(letter , "R2D2")
['R', 'D']
```

Built-In Character Classes

Python regular expressions include the following pre-defines character classes:

- `\d` for a digit and `\D` for the complement of a digit,
- `\s` for whitespace and `\S` for the complement of whitespace.
- `.` for any character except for *newline*.

Escape Characters and Raw Strings

Some features of regular expressions use the backslash character `\`.

Recall that Python already uses backslash as an **escape character** to signify that the next character in a string has a special meaning (e.g. `'\n'` for newline).

In this case, we want backslash to have its usual meaning *as a string* but a special meaning *as a regular expression*.

To do this, we have two options:

- Escape the backslash (e.g. `'\\d'`).
- Use **raw strings**, which have no escaped characters (e.g. `r'\d'`).

Iteration

The second way to build compound regular expressions out of simpler ones is by **iteration**.

To specify a regular expression that represents a given regular expression iterated any (positive) number of times follow it with the special symbol +:

```
number = r'\d+'  # one or more consecutive digits

> re.findall(number , "HAL-9000")
['9000']
```

Notice that by default regular expression matching is **greedy**: it will make the longest match possible, which is often what we want.

Activity: Words

Use *iteration* to define a regular expression `word` that represents any nonempty string of alphabetic characters.

For example:

```
> re.findall(word , "HAL-9000")
['HAL']
```

Fixed Repetition

If we want to repeat a pattern a *fixed* number of times, we indicate this by putting the number between braces `{` and `}`:

```
three_digit_number = r'\d{3}'
```

```
> re.findall(three_digit_number , "HAL-9000")
['900']
```

Notice that by default regular expression matching is **non-overlapping**:
even though `000` is also a three digit number, the first two `0`s are “consumed” by matching `900`. This is not always what we want.

Range Repetition

If we want a range of possible repetitions, we indicate this by putting two number between braces, separated by a comma `,`:

```
three_or_four_digit_number = r'\d{3,4}'
```

```
> re.findall(three_or_four_digit_number , "HAL-9000")
['9000']
```

Notice that by default regular expression matching prefers the longest non-overlapping match.

Optionality

Optionally including a pattern in a regular expression can be implemented as repeating it zero-to-one times:

```
color = 'colou{0,1}r'

re.search(color , "color")
<re.Match object; span=(0, 5), match='color'>
re.search(color , "colour")
<re.Match object; span=(0, 6), match='colour'>
```

There is a shorthand for this using the special symbol `?`:

```
colour = 'colou?r'
```

String Boundaries

Sometimes we want to specify that a regular expression should match only at the beginning or end of a given string.

For this we use the following **metacharacters**:

- `^` matches the *beginning* of a string,
- `$` matches the *end* of a string.

```
only_number = r'^\d+$'
```

```
> re.findall(only_number , "HAL-9000")
[]
> re.findall(only_number , "9000")
['9000']
```

If you want to include a literal `^` or `$` in a regular expression then you need to *escape* it as `\^` or `\$`

Parentheses

Parentheses are used for grouping in regular expressions, like in python:

```
re.search('c|hat' , "cat")
<re.Match object; span=(0, 1), match='c'>
re.search('(c|h)at' , "cat")
<re.Match object; span=(0, 3), match='cat'>
```

If you want to include literal parentheses in a regular expression then you need to escape them as `\(` and `\)`.

Activity: Telephone Numbers

Write a regular expression `telephone` for U.S. telephone numbers in `(XXX)XXX-XXXX` format.

For example:

```
> re.search(telephone , "(222)867-5309")
<re.Match object; span=(0, 13), match='(222)867-5309'>
```

To Do

Finish *Homework 12: More Object-Oriented Programming* on CodeRunner in by next Thursday.

Finish *Project 4: Bioinformatics* on CodeRunner in by the end of the semester (May 8).